

Fun with R*Just another R blog***Network visualization – part 2: Gephi**Posted on [July 26, 2013](#) by [Vessy](#)

In the second part of my “how to quickly visualize networks directly from R” series, I’ll discuss how to use R and the “[rgexf](#)” package to create network plots in [Gephi](#). Gephi is a great network visualization tool that allows real-time network visualization and exploration, including network data spatializing, filtering, calculation of network properties, and clustering. Unfortunately, a current version of the “[rgexf](#)” package does not support real-time network visualization and exploration. However, it allows users to create a .gexf file that contains information about network nodes and edges (and their properties); such file can be loaded and visualized by Gephi.

To demonstrate how this works, I will use the same network I used to demonstrate network visualization in Cytoscape – the weighted network of characters’ coappearances in Victor Hugo’s novel “Les Miserables” ([LesMiserables.txt](#)). I will also use the same node and edge properties: the degree of a node, betweenness centrality of a node, Dice similarity of two nodes, and the coappearance weight. The idea is to use “[rgexf](#)” to create a network visualization in Gephi that captures the network patterns in the similar way the network visualization in “RCytoscape”/Cytoscape did. For more information, see [Network visualization – part 1](#)). See the bottom of this post for full code.

To create a .gexf file, I used “[write.gexf](#)” function. This function creates a gexf representation of a network, based on a user specified data frames describing network nodes and edges. Given the Les Miserables network in the “igraph” format (denoted as “gD”), we can easily create a data frame describing network nodes as:

```
data.frame(ID = c(1:vcount(gD)), NAME = V(gD)$name)
```

and data frame describing network edges as:

```
as.data.frame(get.edges(gD, c(1:ecount(gD))))
```

“[write.gexf](#)” also allows users to assign any type of node and edge attributes to nodes and edges in the network. If we decide to use node/edge attributes, we have to define each attribute for all nodes/edges and the order of attribute values has to correspond to the order of nodes/edges in the edge/node data frames.

We assigned the degree and the betweenness centrality to nodes in our network as:

```
data.frame(DEG = V(gD)$degree, BET = V(gD)$betweenness)
```

Similarly, we assigned the Dice similarity the co-appearance weight to edges in our network as:

```
data.frame(WGH = E(gD)$weight, SIM = E(gD)$similarity)
```

This was easy.

In addition to these (basic) options, the “write.gexf” function provides options to define node and edge visualization attributes, *e.g.*, their color, position, and size/thickness, type of the network (directed or undirected), as well as the edge dynamics. Here, I’ll focus only on visualization options.

Some of the available visualization attributes in “write.gexf” are not supported by the current version of Gephi, *e.g.*, different node and edge shapes within the same network or representation of nodes as images (although, it is great that the “rgexf” authors thought ahead).

First, let’s define node coordinates. The easiest way to do so is with the “igraph” function “layout.” As my goal is to create a similar network plot as I did in Cytoscape, I used a Fruchterman and Reingold’s force-based layout algorithm. Similarly as before, I used the Dice similarity to define the force between two nodes. The “layout.fruchterman.reingold” function can output node layout/coordinates in 2D and 3D. The “write.gexf” function requires that node coordinates are 3D (another think ahead option?). However, Gephi plots networks in 2D and using 3D coordinates for 2D plots will not result in the nicest layout/plot. For that reason, I decided to cheat – create 2D layout/coordinates, and assign 0 as the third coordinate.

Next, I defined node size. To stay consistent with the Cytoscape plot, node size should be interpolated based on the node betweenness centrality. As the “rgexf” library does not provide any options for node size/color interpolation, I had to do it myself. I defined the node size interval as [1, 5] (thus, min node size is 1, max node size is 5). I used the “approx” function to interpolate the node size among n unique betweenness values:

```
approx(c(1, 5), n = length(unique(V(gD)$betweenness)))
```

Then, I assigned the interpolated node size values to each node as:

```
sapply(V(gD)$betweenness, function(x) approxVals$y[which(sort(unique(V(gD)$betweenness)) == x)])
```

Note that this approach scales node size values solely based on the number of unique betweenness centrality values, not based on the ratio of those values.

Similarly, for node color, I needed to define a way to interpolate node colors based on the node degree. To do so, I used the “colorRampPalette” function from the “grDevices” library:

```
F2 <- colorRampPalette(c("#F5DEB3", "#FF0000"),
  bias = length(unique(V(gD)$degree)),
  space = "rgb", interpolate = "linear")
```

This function returns another function (I called it F2), that allows us to create a color palette. Next, we assign a color for each of the degree values:

```
colCodes <- F2(length(unique(V(gD)$degree)))
```

Finally, similarly as before, we used the “sapply” function to map colors to the corresponding nodes:

```
sapply(V(gD)$degree,
  function(x) colCodes[which(sort(unique(V(gD)$degree)) == x)])
```

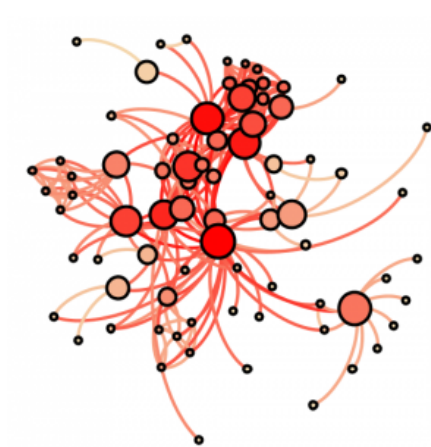
“write.gexf” requires that node colors are presented in the RGBA color mode, *i.e.*, with a 4 coordinates: r, g, b, and alpha; r, g, b are integers between 0 and 256, and alpha is a float between 0.0 and 1.0, corresponding to node transparency. Currently, our colors are in RGB hexadecimal and we need to transform them to RGB decimal. To do so, I’ll use the “col2rgb” function:

```
col2rgb(nodes_col, alpha = FALSE)
```

Although this function can return the alpha value, the value it returns is in [0, 255] and does not match the alpha value required by “write.gexf.” For that reason, I manually assigned alpha = 1 to all node.

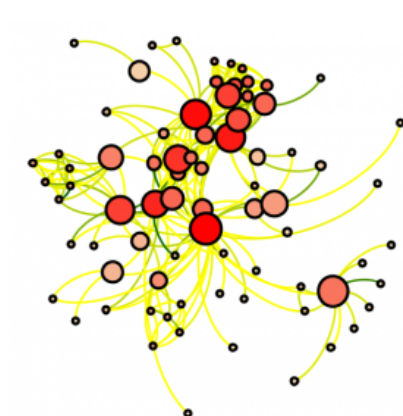
With this we defined all node visual attributes. Similar strategy can be applied to define edge visual attributes and save/write the network. The example of the .gexf network made in this example is available in “[lesmis.gexf](#)” file. So, let’s see how this network looks in Gephi.

When opening the .gexf file, be sure to deselect “Auto-scale” and “Create missing nodes” options in the Gephi Import report. Also, it seems that the “defaultedgetype” options written in the .gexf file was not recognized, so change the “Graph Type” to “Undirected.”

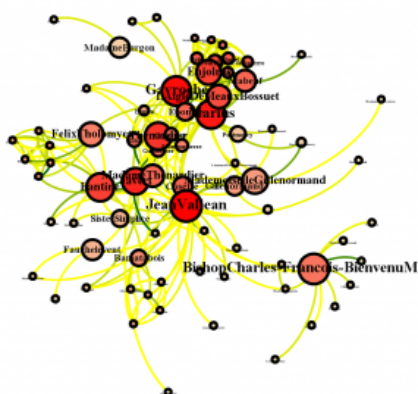


We can see that the network layout fits the force based layout, and that the node size and colors correspond to node properties. However, edge colors do not match those we specified. This does not mean that our color attributes do not exist, but that they were not selected by default. To change this, we have to go to “Preview settings” -> “Edges” -> “Colors” and select “Original.”

And here is our network.

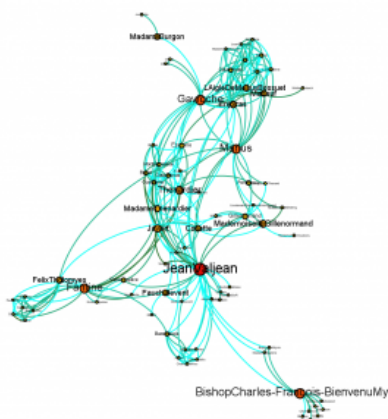


To show node names, we have to go to “Preview settings” -> “Node labels” and check the box “Show labels.” We can scale the labels sizes (to match node sizes) if we check box “Proportional size.” Here is the final version of our network plot.



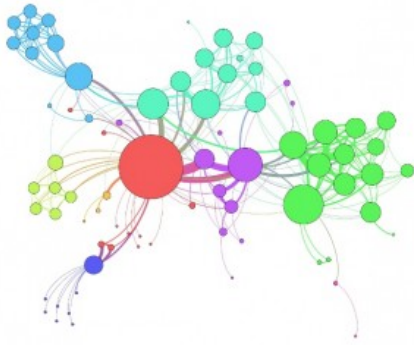
Although the plot shows patterns in node connectivity, many nodes and labels are overlapping with each other. It is clear that in order to get a high-quality plot, we need to make additional adjustments within the Gephi. This raises a question of whether or not user specified visual attributes are an efficient way to define network visual characteristics and how successfully “write.gexf” can translate these attributes into a user specified network plot.

To explore this question, I created a network using only information about network, nodes, edges, and their attributes (thus, no visualization attributes). Then, I used Gephi network visualization options (within Gephi) to create layout and adjust node/edge attributes. In about the same number of steps as it took me to “show” the visual attributes in the previous step, I was able to create the following network plot.



This network is clearly more appealing than the one generated based on the visual attributes specified in the .gexf file. This implies that there is almost no gain in defining user specified visualization attributes in the current version of “rgexf”/“write.gexf,” as the user-specified visualization .gexf network file does not results in “ready-to-use” plots.

If you ever used Gephi you probably noticed that the “Les Miserables” network is one of the network examples that come with Gephi. Here is the Les Miserables network that shows all the visualization power of Gephi. Hopefully, soon we’ll be able to create the same quality network plots directly from R.



Here is the complete code used for the Gephi visualization example.

```

1  # Plotting networks in R
2  # An example how to use R and rgexf package to create a .gexf file for network visualization in Gephi
3
4  #####
5  # Clear workspace
6  rm(list = ls())
7
8  # Load libraries
9  library("igraph")
10 library("plyr")
11
12 # Read a data set.
13 # Data format: dataframe with 3 variables; variables 1 & 2 correspond to interactions; variable 3 corresponds to the weight of interaction
14 dataSet <- read.table("lesmis.txt", header = FALSE, sep = "\t")
15
16 # Create a graph. Use simplify to ensure that there are no duplicated edges or self loops
17 gD <- simplify(graph.data.frame(dataSet, directed=FALSE))
18
19 # Print number of nodes and edges
20 # vcount(gD)
21 # ecount(gD)
22
23 #####
24 # Calculate some node properties and node similarities that will be used to illustrate
25 # different plotting abilities
26
27 # Calculate degree for all nodes
28 degAll <- degree(gD, v = V(gD), mode = "all")
29
30 # Calculate betweenness for all nodes
31 betAll <- betweenness(gD, v = V(gD), directed = FALSE) / (((vcount(gD) - 1) * (vcount(gD) - 2)) / 2)
32 betAll.norm <- (betAll - min(betAll)) / (max(betAll) - min(betAll))
33 rm(betAll)
34
35 # Calculate Dice similarities between all pairs of nodes
36 dsAll <- similarity.dice(gD, vids = V(gD), mode = "all")
37
38 #####
39 # Add new node/edge attributes based on the calculated node properties/similarities
40
41 gD <- set.vertex.attribute(gD, "degree", index = V(gD), value = degAll)
42 gD <- set.vertex.attribute(gD, "betweenness", index = V(gD), value = betAll.norm)
43
44 # Check the attributes
45 # summary(gD)
46
47 F1 <- function(x) {data.frame(V4 = dsAll[which(V(gD)$name == as.character(x$V1)), which(V(gD)$name == as.character(x$V2))])}

```

```

48  dataSet.ext <- ddply(dataSet, .variables=c("V1", "V2", "V3"), function(x) data.frame(F1(x)))
49
50  gD <- set.edge.attribute(gD, "weight", index = E(gD), value = 0)
51  gD <- set.edge.attribute(gD, "similarity", index = E(gD), value = 0)
52
53  # The order of interactions in gD is not the same as it is in dataSet or as it is in the edge list,
54  # and for that reason these values cannot be assigned directly
55
56  E(gD)[as.character(dataSet.ext$V1) %--% as.character(dataSet.ext$V2)]$weight <- as.numeric(dataSet.ext$V3)
57  E(gD)[as.character(dataSet.ext$V1) %--% as.character(dataSet.ext$V2)]$similarity <- as.numeric(dataSet.ext$V4)
58
59
60  # Check the attributes
61  # summary(gD)
62
63  #####
64  # Print network in the file format ready for Gephi
65  # This requires rgexf package
66
67  library("rgexf")
68
69  # Create a dataframe nodes: 1st column - node ID, 2nd column -node name
70  nodes_df <- data.frame(ID = c(1:vcount(gD)), NAME = V(gD)$name)
71  # Create a dataframe edges: 1st column - source node ID, 2nd column -target node ID
72  edges_df <- as.data.frame(get.edges(gD, c(1:ecount(gD))))
73
74  # Define node and edge attributes - these attributes won't be directly used for network visualization, but they
75  # may be useful for other network manipulations in Gephi
76  #
77  # Create a dataframe with node attributes: 1st column - attribute 1 (degree), 2nd column - attribute 2 (betweenness)
78  nodes_att <- data.frame(DEG = V(gD)$degree, BET = V(gD)$betweenness)
79  #
80  # Create a dataframe with edge attributes: 1st column - attribute 1 (weight), 2nd column - attribute 2 (similarity)
81  edges_att <- data.frame(WGH = E(gD)$weight, SIM = E(gD)$similarity)
82
83  # Define node/edge visual attributes - these attributes are the ones used for network visualization
84  #
85  # Calculate node coordinate - needs to be 3D
86  #nodes_coord <- as.data.frame(layout.fruchterman.reingold(gD, weights = E(gD)$similarity, dim = 3, niter = 10000))
87  # We'll cheat here, as 2D coordinates result in a better (2D) plot than 3D coordinates
88  nodes_coord <- as.data.frame(layout.fruchterman.reingold(gD, weights = E(gD)$similarity, dim = 2, niter = 10000))
89  nodes_coord <- cbind(nodes_coord, rep(0, times = nrow(nodes_coord)))
90  #
91  # Calculate node size
92  # We'll interpolate node size based on the node betweenness centrality, using the "approx" function
93  approxVals <- approx(c(1, 5), n = length(unique(V(gD)$betweenness)))
94  # And we will assign a node size for each node based on its betweenness centrality
95  nodes_size <- sapply(V(gD)$betweenness, function(x) approxVals$y[which(sort(unique(V(gD)$betweenness)) == x)])
96  #
97  # Define node color
98  # We'll interpolate node colors based on the node degree using the "colorRampPalette" function from the "grDevices" library
99  library("grDevices")
100 # This function returns a function corresponding to a color palette of "bias" number of elements
101 F2 <- colorRampPalette(c("#F5DEB3", "#FF0000"), bias = length(unique(V(gD)$degree)), space = "rgb", interpolate = "linear")
102 # Now we'll create a color for each degree
103 colCodes <- F2(length(unique(V(gD)$degree)))
104 # And we will assign a color for each node based on its degree
105 nodes_col <- sapply(V(gD)$degree, function(x) colCodes[which(sort(unique(V(gD)$degree)) == x)])
106 # Transform it into a data frame (we have to transpose it first)
107 nodes_col_df <- as.data.frame(t(col2rgb(nodes_col, alpha = FALSE)))
108 # And add alpha (between 0 and 1). The alpha from "col2rgb" function takes values from 0-255, so we cannot use it
109 nodes_col_df <- cbind(nodes_col_df, alpha = rep(1, times = nrow(nodes_col_df)))

```

```

110 # Assign visual attributes to nodes (colors have to be 4dimensional - RGBA)
111 nodes_att_viz <- list(color = nodes_col_df, position = nodes_coord, size = nodes_size)
112
113 # Assign visual attributes to edges using the same approach as we did for nodes
114 F2 <- colorRampPalette(c("#FFFF00", "#006400"), bias = length(unique(E(gD)$weight)), space = "rgb", interpolate = "linear")
115 colCodes <- F2(length(unique(E(gD)$weight)))
116 edges_col <- sapply(E(gD)$weight, function(x) colCodes[which(sort(unique(E(gD)$weight)) == x)])
117 edges_col_df <- as.data.frame(t(col2rgb(edges_col, alpha = FALSE)))
118 edges_col_df <- cbind(edges_col_df, alpha = rep(1, times = nrow(edges_col_df)))
119 edges_att_viz <-list(color = edges_col_df)
120
121 # Write the network into a gexf (Gephi) file
122 #write.gexf(nodes = nodes_df, edges = edges_df, nodesAtt = nodes_att, edgesWeight = E(gD)$weight, edgesAtt = edges_att, nodesVizAtt = r
123 # And without edge weights
124 write.gexf(nodes = nodes_df, edges = edges_df, nodesAtt = nodes_att, edgesAtt = edges_att, nodesVizAtt = nodes_att_viz, edgesVizAtt = e

```

plotNetworksUsingGephi.R hosted with ❤ by GitHub [view raw](#)

This entry was posted in [Graphs](#), [Networks](#), [Visualization](#) and tagged [Gephi](#), [Networks/Graphs](#), [rgexf](#), [Visualization](#). Bookmark the [permalink](#).

7 Responses to Network visualization – part 2: Gephi



bibzzzz says:

October 2, 2013 at 8:14 am

The order of interactions in gD is not the same as it is in dataSet or as it is in the edge list,
and for that reason these values cannot be assigned directly

```

for (i in 1:nrow(dataSet.ext))
{
  E(gD)[as.character(dataSet.ext$V1) %--% as.character(dataSet.ext$V2)]$weight <- as.numeric(dataSet.ext$V3)
  E(gD)[as.character(dataSet.ext$V1) %--% as.character(dataSet.ext$V2)]$similarity <- as.numeric(dataSet.ext$V4)
}

```

Are you sure this loop is set up properly?

PS. Thanks for sharing – this is excellent.

[Reply](#)



admin says:

October 4, 2013 at 12:48 am

Thanks for noticing it!



The loop was useless, as the values can be (are) assigned directly to edges. Here is the fix (also included in the gist):

```

E(gD)[as.character(dataSet.ext$V1) %--% as.character(dataSet.ext$V2)]$weight <- as.numeric(dataSet.ext$V3)
E(gD)[as.character(dataSet.ext$V1) %--% as.character(dataSet.ext$V2)]$similarity <- as.numeric(dataSet.ext$V4)

```

[Reply](#)



George Chacko says:

March 8, 2015 at 6:59 pm

rgexf isn't on CRAN- and a version compatible with R 3.12 doesn't seem to be available. Are there alternate instructions to install? Thanks

[Reply](#)



itsab says:

December 21, 2016 at 2:07 pm

```
source("https://bioconductor.org/biocLite.R")
biocLite("rgexf")
```

best regards 😊

[Reply](#)



haiyuan says:

March 10, 2015 at 7:17 am

package 'rgexf' is not available (for R version 3.1.0) , other package can be replaced with "rgexf"? thanks

[Reply](#)



itsab says:

December 21, 2016 at 2:08 pm

```
source("https://bioconductor.org/biocLite.R")
biocLite("rgexf")
```

best regards 😊

[Reply](#)



admin says:

March 17, 2015 at 9:06 pm

According to the wiki page (<https://bitbucket.org/gvegayon/rgexf/wiki/Home>), a new version (Version 0.15.2.3) of rgexf is on CRAN as of 2015-02-03. However, I was not able to install it either (did not exist).

An alternative way would be to try to install it from the Archive (<http://cran.r-project.org/src/contrib/Archive/rgexf/>), but I am not sure if that will resolve the compatibility issues. Probably the best thing to do is to contact the package authors and ask them directly about this issue (you can find their email addresses in the "description" file in the archived packages or you can report an issue at <https://bitbucket.org/gvegayon/rgexf/issues?status=new&status=open> – looks like somebody reported it already, but it was not resolved).

Regarding other packages/programs that can replace "rgexf," I'd suggest Cytoscape. You can check my post about the "RCytoscape" package and Bill Longabaugh's comment about how to communicate with Cytoscape with newer versions of R.

Hope this helps!

[Reply](#)